



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 3

Team No.: 2

Project Title: CPU Scheduling system

Team Members

Roll Number	Student Name	Signature
2520030309	Nuguri Siri	
2520030485	M.Geethika	

1. Abstract

The CPU Scheduling Simulation System is a Linux-based C project designed to demonstrate how an operating system manages multiple processes competing for CPU time. The project addresses the problem of selecting and scheduling processes efficiently while analyzing their performance. The system implements FCFS, SJF, Priority, and Round Robin scheduling algorithms and maintains process states such as ready, running, waiting, and terminated. It uses pipes and named pipes (FIFOs) for communication between system components and signals to simulate timer interrupts and time-slice expiration. Dynamic memory allocation is used to maintain process-control information during execution. The system calculates important scheduling metrics such as waiting time, turnaround time, and response time, and displays the scheduling order and performance statistics. The proposed solution will be implemented in C on a Linux environment using Linux/POSIX system-programming concepts. The expected outcome is a functional scheduling simulator that helps users understand and compare different CPU scheduling techniques and observe their effect on process execution and performance.

2. Problem Statement

The project focuses on the problem of managing multiple processes that compete for CPU time in an operating system. Since different processes require different amounts of CPU time, an appropriate scheduling algorithm is needed for efficient process management. The system will simulate and compare FCFS, SJF, Priority, and Round Robin scheduling algorithms and track process states such as ready, running, waiting, and terminated. It will also demonstrate pipes, named pipes (FIFOs), signals, and dynamic memory allocation for process communication and scheduling activities. This project is relevant to Operating Systems and Systems Programming as it demonstrates how processes are scheduled, managed, synchronized, and communicated in a Linux environment.

3. Objectives

- To understand and implement different CPU scheduling algorithms.
- To simulate FCFS, SJF, Priority, and Round Robin scheduling.
- To manage and display different process states such as Ready, Running, Waiting, and Terminated.
- To calculate Waiting Time, Turnaround Time, and Completion Time.
- To compare the performance of different scheduling algorithms.
- To demonstrate basic OS concepts such as process management, pipes, signals, and dynamic memory allocation.

4. Proposed Methodology

The project will be designed and implemented in a **Linux environment using C programming**. The system will accept process details such as **Process ID, Arrival Time, Burst Time, and Priority** as input. It will apply **FCFS, SJF, Priority, and Round Robin** scheduling algorithms to decide the order of process execution. The process flow will be displayed using states such as **Ready, Running, Waiting, and Terminated**. Linux concepts such as **process creation, pipes, signals, and dynamic memory allocation** will be used to demonstrate OS functionality. Finally, the system will calculate **Waiting Time, Turnaround Time, and Completion Time** and compare the performance of all scheduling algorithms.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Management	To create, manage, and schedule processes
fork()	To create child processes for simulation
pipe()	To enable communication between processes.
CPU Scheduling	To implement FCFS, SJF, Priority, and Round Robin.
Process States	To represent Ready, Running, Waiting.
wait()	To allow a parent process to wait for child process.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030309	Nuguri Siri	https://github.com/siripanchajanya-

		netizen/CPU-Scheduling-Simulation-System
2520030485	M.Geethika	https://github.com/geethika04-dot/CPU-Scheduling-System

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system environment for developing and running the project.
C / C++	Used to implement CPU scheduling algorithms and OS concepts
GCC Compiler	Used to compile and execute the C/C++ programs.
Linux System Calls / POSIX APIs	Used for process management, pipes, signals, and process synchronization.
Terminal	Used to compile, run, and test the project.

8. Expected Outcome

- The system will successfully simulate **FCFS, SJF, Priority, and Round Robin** scheduling algorithms.
- It will display the **execution order and process states** such as Ready, Running, Waiting, and Terminated.
- It will calculate **Waiting Time, Turnaround Time, and Completion Time** for each process.
- It will provide a **comparison of the scheduling algorithms** based on their performance.
- It will demonstrate Linux OS concepts such as **process management, pipes, signals, and dynamic memory allocation**.
- The project will help users understand how different CPU scheduling algorithms affect **CPU utilization and process performance**.

9. GitHub Repository

GitHub Repository Link: <https://github.com/siripanchajanya-netizen/CPU-Scheduling-Simulation-System>

Repository Created: YES

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Dr.K.Hema

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____